RAPPORT DE TRAVAUX PRATIQUES

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BÉNI MELLAL

TP1 – Application Monolithique Spring Boot

Réalisé par : EL MELIANI Ammar
Encadrant : Pr. EL BAGHAZAOUI

Filière : **IACS-S2**
Année universitaire : 2025/2026

Lien GITHUB :

https://github.com/EL-Star456/TP_Rapport_Log.git

TP 1 - Application Monolithique avec Spring Boot

1. Objectifs du TP

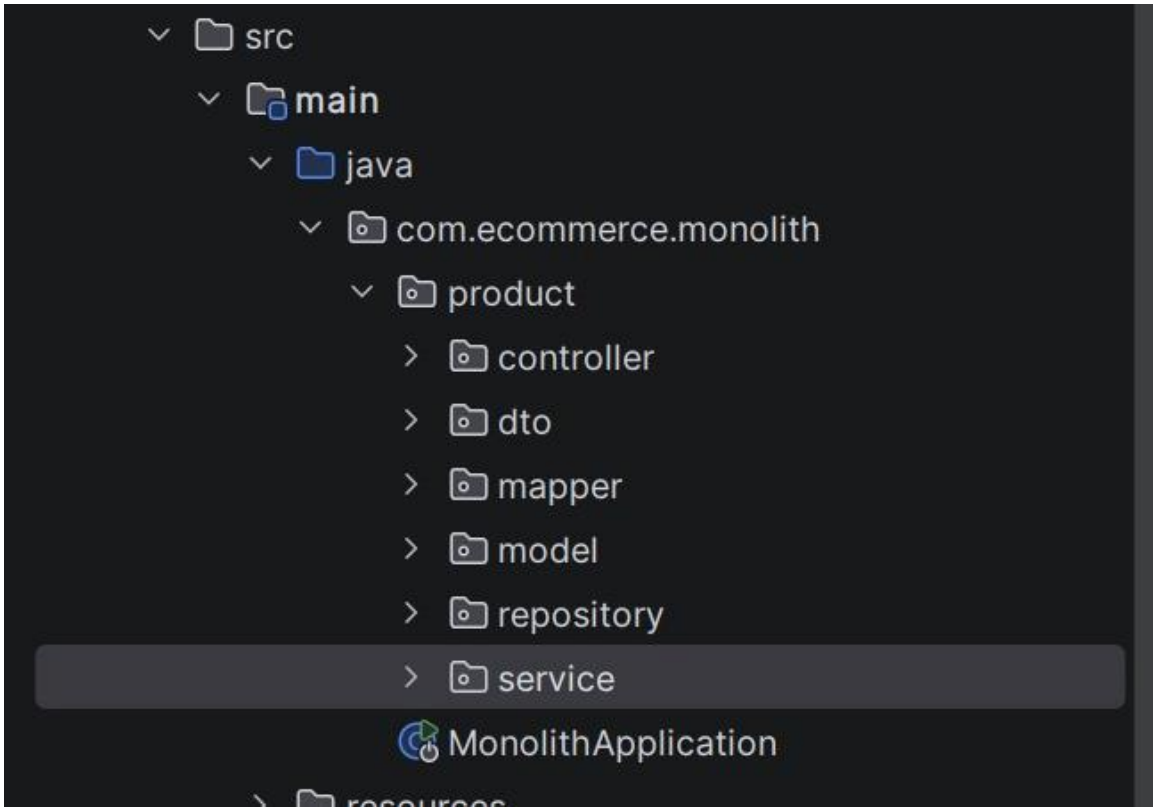
- **Créer une application Spring Boot monolithique simple**
- **Développer une API REST CRUD complète**
- **Comprendre l'architecture en couches (Entity, Repository, Service, Controller)**
- **Découvrir JPA et Hibernate pour la persistance des données**
- **Identifier les avantages et limites d'un monolithe**

2. Architecture du Projet

L'application suit une architecture en couches (Layered Architecture) classique, composée de quatre couches distinctes :

2.1 Structure des packages

```
com.ecommerce.monolith
  product/
    controller/    -> ProductController.java
    service/       -> ProductService.java
    repository/    -> ProductRepository.java
    model/         -> Product.java, Category.java
```



2.2 Description des couches

Controller : Point d'entree de l'API REST. Recoit les requetes HTTP (GET, POST, PUT, DELETE) et retourne des reponses JSON avec les codes HTTP appropries (200, 201, 204).

Service : Contient la logique metier de l'application. Fait le lien entre le Controller et le Repository. Annote avec @Service et @Transactional pour la gestion automatique des transactions.

Repository : Interface d'acces a la base de donnees. Etend JpaRepository qui genere automatiquement toutes les requetes SQL (SELECT, INSERT, UPDATE, DELETE) sans ecrire une seule ligne de SQL.

Model (Entity) : Representation des donnees en Java. Correspond directement aux tables PostgreSQL via les annotations JPA (@Entity, @Table, @Column).

3. Implémentation

3.1 Entité Product

L'entite Product represente un produit dans la base de donnees. Elle utilise les annotations JPA pour le mapping objet-relationnel et les annotations Lombok pour reduire le code repetitif.

3.2 Repository

Le repository etend JpaRepository<Product, Long> qui fournit automatiquement les methodes CRUD. Une methode de recherche par nom est ajoutee pour illustrer les query methods de Spring Data.

3.3 Service

Le service contient la logique metier avec la gestion des transactions. L'annotation @Transactional(readOnly = true) optimise les requetes de lecture, et @Transactional gere automatiquement les commits et rollbacks.

3.4 Controller

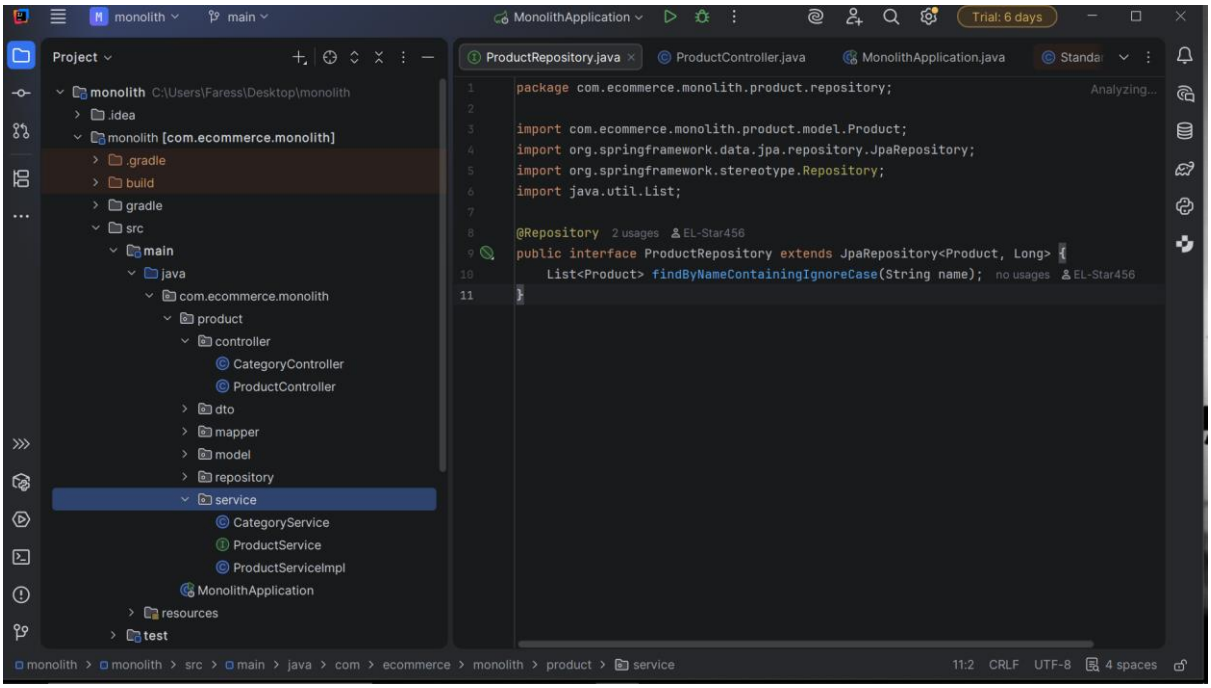
Le controller expose les endpoints REST avec les annotations Spring MVC. Chaque methode HTTP correspond a une operation CRUD :

Methode HTTP	URL	Description	Code Reponse
GET	/api/products	Lister tous les produits	200 OK
GET	/api/products/{id}	Obtenir un produit par ID	200 OK
POST	/api/products	Creer un nouveau produit	201 Created
PUT	/api/products/{id}	Modifier un produit	200 OK
DELETE	/api/products/{id}	Supprimer un produit	204 No Content

4. Exercice - Entite Category

En bonus, une entite Category a ete ajoutee avec une relation ManyToOne avec Product. Cela illustre les relations entre entites JPA et la creation automatique des tables avec les contraintes de cles etrangeres.

Hibernate a genere automatiquement la table categories et la cle etrangere category_id dans la table products, sans aucune intervention manuelle en SQL.



6. Tests avec Postman

Les differents endpoints ont ete testes avec Postman. Voici les resultats obtenus :

Test	Methode	URL	Resultat
Creer un produit	POST	/api/products	201 Created - OK
Lister les produits	GET	/api/products	200 OK - Liste vide puis avec donnees
Modifier un produit	PUT	/api/products/1	200 OK - Produit mis a jour
Supprimer un produit	DELETE	/api/products/1	204 No Content - OK
Creer une categorie	POST	/api/categories	201 Created - OK
Produit avec categorie	POST	/api/products	201 Created avec category_id

POST http://localhost:8080/ GET http://localhost:8080/a POST http://localhost:8080/ No environ

http://localhost:8080/api/products Save

POST http://localhost:8080/api/products Send

Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2     "name": "Laptop Dell",
3     "description": "Ordinateur portable",
4     "price": 1299.99,
5     "stock": 10
6 }
```

Body 201 Created 968 ms 261 B

Pretty Raw Preview Visualize JSON

```
1 {
2     "id": 4,
3     "name": "Laptop Dell",
4     "description": "Ordinateur portable",
5     "price": 1299.99,
6     "stock": 10
7 }
```

Body 200 OK 311 ms 444 B

Pretty Raw Preview Visualize JSON

```
1 [
2     {
3         "id": 2,
4         "name": "Laptop Dell",
5         "description": "Ordinateur portable",
6         "price": 1299.99,
7         "stock": 10
8     },
9     {
10        "id": 3,
11        "name": "Laptop Dell",
12        "description": "Ordinateur portable",
13        "price": 1299.99,
14        "stock": 10
15    },
```

7. Conclusion TP1:

Le TP1 a permis de construire une API REST Spring Boot complete avec connexion PostgreSQL. L'architecture en couches (Controller, Service, Repository, Entity) est le standard professionnel utilise dans l'industrie. La puissance de Spring Boot reside dans sa capacite a generer automatiquement le code repetitif (requetes SQL, endpoints CRUD) grace a Spring Data JPA et aux annotations.

Les limites du monolithe simple sont visibles : tout le code est dans un seul module, ce qui devient difficile a maintenir quand le projet grandit. C'est pourquoi le TP2 introduit le concept de monolithe modulaire.

Fin